

Lab10 Report

22300240022 王镜凯

实现了一个较为简单的编译器优化器，主要进行常量传播（Constant Propagation）和死代码消除（Dead Code Elimination）优化。

核心数据结构

从 opt.hh 文件可以看到，优化器使用了以下关键数据结构：

1. RtValue 类：表示变量的运行时值，有三种状态：
 - NO_VALUE：未定义值
 - ONE_VALUE：确定的常量值
 - MANY_VALUES：不确定的值（可能有多个可能的值）
2. Opt 类：主要优化器类，包含：
 - block_executable：记录基本块是否可执行
 - temp_value：记录临时变量的值状态
 - label2block：标签到基本块的映射

优化流程

整个优化过程分为三个主要步骤：

1. 初始化（optFunc 方法）

- 初始化所有基本块为不可执行
- 建立标签到基本块的映射
- 将所有函数参数初始化为 MANY_VALUES（不确定值）

2. 常量传播分析（calculateBT 方法）

这是一个数据流分析过程，使用迭代算法：

- 初始化：
 - 所有变量设为 MANY_VALUES（顶元素 $\top$ ）
 - 将第一个基本块标记为可执行
- 迭代过程：
 - 遍历所有可执行的基本块
 - 对每个基本块中的四元式指令进行分析：
 - MOVE 指令：分析源操作数，更新目标变量的值
 - MOVE_BINOP 指令：如果两个操作数都是常量，计算结果并更新目标变量

- PHI 指令：分析所有可执行路径的值，如果都相同则更新目标变量
 - 内存访问和函数调用：将定义的变量设为 MANY_VALUES
 - 跳转指令：更新可执行基本块信息
- 重复直到没有变化

3. 代码优化（modifyFunc 方法）

根据分析结果修改代码：

- 移除不可执行的基本块
- 对每个可执行基本块中的指令：
 - 如果目标变量是常量，移除赋值指令
 - 将使用常量的操作数替换为常量值
 - 优化条件跳转：如果条件是常量，替换为无条件跳转
 - 优化 PHI 指令：为常量参数创建新的赋值指令

具体实现细节

常量传播分析

代码使用迭代数据流分析算法，主要处理以下几种指令：

1. 赋值指令（MOVE）：

```
if (quad->kind == QuadKind::MOVE) {
    auto move = static_cast<QuadMove*>
        (quad);
    auto dest = move->dst->temp->num;
    // 获取源操作数的值
    RtValue srcValue = src_is_const ?
        RtValue(src_val) : getRtValue
            (src_val);
    // 更新目标变量的值
    if (srcValue.getType() !=
        temp_value[dest].getType() ||
        (srcValue.getType() ==
            ValueType::ONE_VALUE &&
            srcValue.getIntValue() !=
                temp_value[dest].getIntValue())) {
        temp_value[dest] = srcValue;
        changed = true;
    }
}
```

2. 二元运算指令（MOVE_BINOP）：

- 如果两个操作数都是常量，直接计算结果

- 否则，将目标设为 MANY_VALUES

3. PHI 指令：

- 检查所有可执行路径的值
- 如果都是相同的常量，则目标变量也是该常量
- 否则，设为 MANY_VALUES

4. 条件跳转 (CJUMP)：

- 如果条件操作数都是常量，计算条件结果
- 根据结果更新可执行基本块

代码优化

优化过程主要包括：

1. 移除不可执行代码：

```
if (!block_executable
[block->entry_label->num]) {
    continue; // 跳过不可执行的基本块
}
```

2. 常量替换：

```
if (temp_value[dest].getType() == ValueType::ONE_VALUE) {
    // 跳过赋值指令，因为目标是常量
} else {
    // 检查操作数是否为常量，如果是则替换
    if (binop->left->get_temp() &&
        temp_value[binop->left->get_temp()->temp->num].getType() ==
        ValueType::ONE_VALUE) {
        binop->use->erase(binop->left->get_temp()->temp);
        binop->left = new QuadTerm(temp_value[binop->left->get_temp()->temp-
>num].getIntValue());
    }
}
```

3. 优化条件跳转：

```
if (val1.getType() == ValueType::ONE_VALUE && val2.getType() == ValueType::ONE_VALUE)
{
    bool condition = false;
    // 计算条件结果
    if (cjump->relop == "<=")
        condition = val1.getIntValue() <= val2.getIntValue();
    // ...

    // 创建无条件跳转替换条件跳转
    QuadJump* new_quad = new QuadJump
```

```

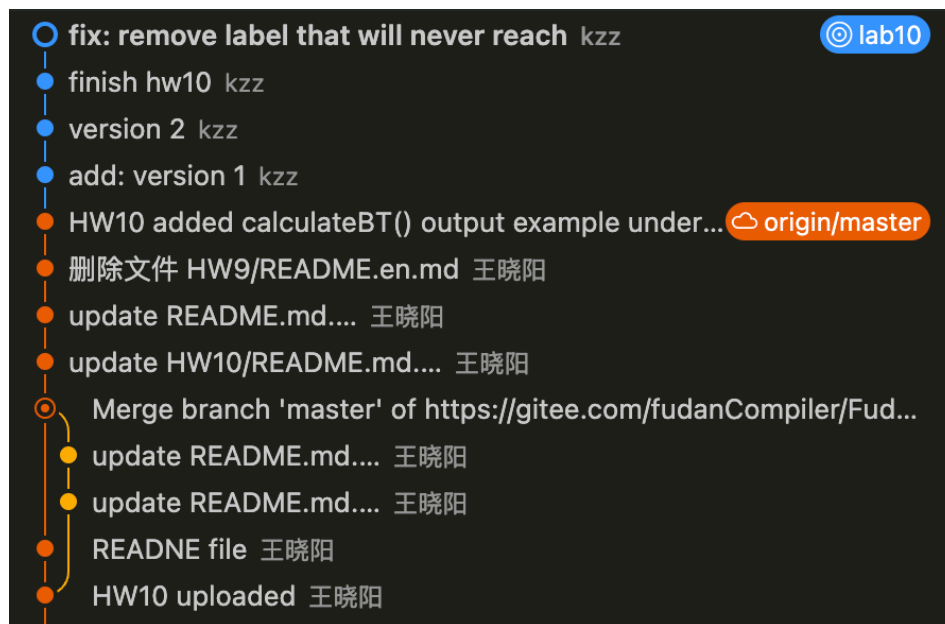
(cjump->node,
 condition ? cjump->t :
 cjump->f,
 def, use);
new_quads->push_back(new_quad);
}

```

4. PHI 指令特殊处理（如果PHI中有ONE_VALUE）：

- 为常量参数创建新的赋值指令，插入到前驱基本块中
- 更新 PHI 指令使用新创建的临时变量

Git开发图



参考资料

- PPT
- 虎书